

Entendendo Ponteiros em Go: Um Guia para Iniciantes

Por Antônio Marcelo

Introdução

Quando começamos a programar em Go, trabalhamos constantemente com variáveis:

```
nome := "Antonio"
idade := 45
```

Normalmente pensamos apenas no valor armazenado na variável. Porém, internamente, esses valores ocupam posições na memória do computador.

Um ponteiro é uma variável especial que armazena o endereço de memória de outra variável.

Embora o assunto assuste muitos iniciantes, os ponteiros em Go são muito mais simples do que em linguagens como C e C++.

O que é um ponteiro?

Imagine que sua casa possui um endereço:

```
Rua das Flores, 100
```

O endereço não é a casa.

O endereço apenas informa onde a casa está localizada.

Com ponteiros acontece exatamente a mesma coisa.

Temos uma variável:

```
idade := 20
```

E temos um ponteiro:

```
p := &idade
```

O ponteiro não guarda o valor 20.

Ele guarda o local onde o valor 20 está armazenado.

O operador &

O símbolo:

```
&
```

significa:

"Me dê o endereço dessa variável."

Exemplo:

```
package main

import "fmt"

func main() {
```

```
idade := 20

fmt.Println(idade)
fmt.Println(&idade)

}
```

Saída:

```
20
0xc0000120a8
```

O número estranho é o endereço da variável na memória.

Declarando um ponteiro

Podemos armazenar esse endereço em outra variável:

```
package main

import "fmt"

func main() {

    idade := 20

    p := &idade

    fmt.Println(p)

}
```

Visualmente:

```
idade = 20

p —————> idade
```

O ponteiro aponta para a variável.

O operador *

O símbolo:

*

significa:

"Acesse o valor armazenado no endereço apontado."

Exemplo:

```
package main

import "fmt"

func main() {

    idade := 20

    p := &idade
```

```
        fmt.Println(*p)
    }
```

Saída:

20

Observe:

p

mostra o endereço.

Já:

*p

mostra o valor armazenado naquele endereço.

Alterando valores usando ponteiros

Uma das maiores vantagens dos ponteiros é modificar uma variável sem acessá-la diretamente.

```
package main

import "fmt"

func main() {
    idade := 20
    p := &idade
    *p = 30
    fmt.Println(idade)
}
```

Saída:

30

Mesmo alterando:

*p

o valor de:

idade

foi modificado.

Isso acontece porque ambos estão apontando para a mesma posição na memória.

Por que usar ponteiros?

A principal razão é evitar cópias desnecessárias.

Considere uma struct:

```
type Aluno struct {
```

```
        Nome string
        Idade int
    }
```

Sem ponteiro:

```
func alterar(aluno Aluno) {
    aluno.Nome = "Maria"
}
```

Ao chamar:

```
alterar(aluno)
```

o Go cria uma cópia da struct.

O objeto original não será alterado.

Exemplo sem ponteiro

```
package main

import "fmt"

type Aluno struct {
    Nome string
}

func alterar(a Aluno) {
    a.Nome = "Maria"
}

func main() {

    aluno := Aluno{
        Nome: "Antonio",
    }

    alterar(aluno)

    fmt.Println(aluno.Nome)

}
```

Saída:

```
Antonio
```

Nada mudou.

Exemplo com ponteiro

```
package main

import "fmt"

type Aluno struct {
    Nome string
}

func alterar(a *Aluno) {
    a.Nome = "Maria"
}
```

```

}

func main() {

    aluno := Aluno{
        Nome: "Antonio",
    }

    alterar(&aluno)

    fmt.Println(aluno.Nome)

}

```

Saída:

Maria

Agora a alteração aconteceu diretamente no objeto original.

Ponteiros em métodos

É muito comum utilizar ponteiros em métodos.

```

type Conta struct {
    Saldo float64
}

```

Método sem ponteiro:

```

func (c Conta) Depositar(valor float64) {
    c.Saldo += valor
}

```

Isso altera apenas uma cópia.

Método com ponteiro:

```

func (c *Conta) Depositar(valor float64) {
    c.Saldo += valor
}

```

Agora o saldo real é alterado.

Exemplo prático: Conta Bancária

```

package main

import "fmt"

type Conta struct {
    Saldo float64
}

func (c *Conta) Depositar(valor float64) {
    c.Saldo += valor
}

func main() {

    conta := Conta{
        Saldo: 100,
    }
}

```

```
    }

    conta.Depositar(50)

    fmt.Println("Saldo:", conta.Saldo)
}
```

Saída:

Saldo: 150

Quando usar ponteiros?

Use ponteiros quando:

- Precisar alterar o objeto original.
- Trabalhar com structs grandes.
- Quiser evitar cópias desnecessárias.
- Criar métodos que modificam o estado do objeto.

Quando NÃO usar ponteiros?

Evite ponteiros quando:

- O valor é pequeno.
- Você não pretende modificar o dado original.
- O código fica mais simples utilizando cópia por valor.

Resumo

Os dois operadores mais importantes são:

&

Obtém o endereço de uma variável.

*

Acessa o valor armazenado naquele endereço.

Visualmente:

```
idade = 20
```

```
p = &idade
```

```
p → idade
```

```
*p = 20
```

Em Go, ponteiros são utilizados principalmente para permitir que funções e métodos alterem objetos originais sem criar cópias desnecessárias, tornando o código mais eficiente e economizando memória.